SecurePlan Mobile — Calendario

Documentación técnica de los Sprints 1 y 2

Reporte de implementación: arquitectura, patrón Singleton, base de datos, seguridad (bcrypt, JWT, IDOR), navegación protegida, comandos utilizados y roadmap de evolución de la base de datos.

Proyecto	SecurePlan Mobile (Calendario)
Asignatura	Desarrollo Móvil Integral
Stack	React Native + Expo SDK 58 / Node.js + Express 5 / PostgreSQL 16
Alcance del reporte	Sprint 1 (base + autenticación) y Sprint 2 (agenda)
Fecha de generación	2026-10-15

Índice de contenido

1. Resumen ejecutivo
2. Objetivo del documento
3. Arquitectura general del sistema
4. La base de datos: cómo fue construida
5. La base de datos: cómo cambiará a futuro
6. Sprint 1 — Base móvil, arquitectura y autenticación segura
7. Patrón Singleton en el proyecto (¿dónde está?)
8. Arquitectura limpia del frontend
9. Navegación protegida (pilas separadas)
10. Sprint 2 — Core de la agenda (módulo de eventos)
11. Codificación segura aplicada
12. Comandos utilizados
13. Reporte de esta versión del documento (qué se agregó y dónde)
14. Roadmap de evolución de la base de datos
15. Conclusión

1. Resumen ejecutivo

SecurePlan Mobile (Calendario) es una aplicación móvil de gestión de agenda construida con una arquitectura desacoplada: un **frontend móvil** (React Native + Expo) y un **backend API REST** (Node.js + Express) que persiste la información en **PostgreSQL**. El proyecto aplica patrones de diseño (Singleton), principios de arquitectura limpia y medidas de codificación segura orientadas a mitigar las vulnerabilidades más comunes en entornos móviles: almacenamiento de credenciales, exposición de tokens y acceso indebido a datos de otros usuarios (IDOR).

Este documento narra, de forma ordenada, lo solicitado por los Sprints 1 y 2 del plan del proyecto y **dónde** fue implementado cada elemento en el código. Además incorpora una sección específica sobre la **base de datos** (cómo fue realizada y cómo se planea cambiar a futuro) y un **reporte de versión** que indica qué se agregó a este documento y en qué apartado.

2. Objetivo del documento

Servir como memo técnica del equipo: para cada requisito de los Sprints 1 y 2, se indica (a) qué pedía el enunciado, (b) qué se hizo, (c) los archivos y funciones concretas donde se realizó, y (d) los comandos empleados. De este modo, cualquier integrante puede rastrear una decisión hasta su punto exacto en el repositorio.

Convención de referencias: las rutas se escriben relativas a la raíz del repositorio. Por ejemplo `backend/src/config/database.js`.

3. Arquitectura general del sistema

El sistema se divide en dos aplicaciones independientes que se comunican por HTTP/JSON. El backend expone una API REST en el puerto 4000; el frontend consume esa API mediante un cliente axios configurado por la variable de entorno `EXPO_PUBLIC_API_URL`.

```
Calendario/
■■■ backend/                 API REST (Node + Express + PostgreSQL)
■   ■■■ src/config/          entorno + conexión a BD (Singleton)
■   ■■■ src/controllers/     lógica de los endpoints
■   ■■■ src/middleware/      autenticación JWT + manejo de errores
■   ■■■ src/models/          acceso a datos + hashing
■   ■■■ src/routes/          definición de rutas
■   ■■■ src/services/        servicios (JWT)
■   ■■■ src/validators/      esquemas de validación (Zod)
■   ■■■ sql/init.sql         esquema de base de datos
■   ■■■ docker-compose.yml   PostgreSQL en desarrollo
■■■ frontend/                App móvil (React Native + Expo)
    ■■■ src/
        ■■■ domain/          entidades, constantes y utilidades puras
        ■■■ data/            cliente HTTP + repositorios
        ■■■ navigation/      navegación protegida (auth / no auth)
        ■■■ presentation/    pantallas, componentes y contexto
        ■■■ services/        almacenamiento seguro (SecureStore)
        ■■■ theme/           tokens de diseño (colores, espaciado)
```

Esta separación permite que el frontend no conozca detalles de la base de datos y que el backend no conozca detalles de la interfaz: el contrato entre ambos es la API REST.

4. La base de datos: cómo fue construida

4.1. Motor y entorno de desarrollo

Se eligió **PostgreSQL 16** por su robustez relacional, tipado fuerte y soporte de transacciones. En desarrollo se levanta mediante **Docker Compose**, lo que garantiza un entorno reproducible y aislado.

El servicio se define en un archivo de composición y utiliza un volumen nombrado para no perder los datos entre reinicios:

```
# backend/docker-compose.yml
services:
  db:
    image: postgres:16-alpine
    container_name: calendario-db
    environment:
      POSTGRES_USER: calendario
      POSTGRES_PASSWORD: calendario
      POSTGRES_DB: calendario
    ports: ["5432:5432"]
    volumes:
      - calendario-pgdata:/var/lib/postgresql/data
      - ./sql/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
```

Un aspecto clave: el archivo `sql/init.sql` se monta en `/docker-entrypoint-initdb.d/`, el directorio que PostgreSQL ejecuta automáticamente la primera vez que se inicializa el volumen. Así, el esquema se crea solo con levantar el contenedor, sin pasos manuales.

4.2. Esquema: tablas e índices

El esquema consta de dos tablas. La tabla **users** guarda las cuentas (el password siempre hasheado, nunca en texto plano) y la tabla **events** guarda la agenda, relacionada con su propietario mediante una clave foránea con borrado en cascada. Se incluyen dos índices para acelerar las consultas más frecuentes: búsqueda de usuario por correo (case-insensitive) y listado de eventos por usuario y fecha.

```
-- backend/sql/init.sql
CREATE TABLE IF NOT EXISTS users (
  id SERIAL PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS idx_users_email ON users (LOWER(email));

CREATE TABLE IF NOT EXISTS events (
  id SERIAL PRIMARY KEY,
  user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  title VARCHAR(200) NOT NULL,
  description TEXT NOT NULL DEFAULT '',
  event_date DATE NOT NULL,
  start_time TIME NOT NULL,
  end_time TIME NOT NULL,
  category VARCHAR(20) NOT NULL DEFAULT 'other',
  priority VARCHAR(20) NOT NULL DEFAULT 'medium',
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS idx_events_user_date ON events (user_id, event_date);
```

Elemento	Propósito	Ubicación

users.email UNIQUE	Evita cuentas duplicadas	sql/init.sql:3
LOWER(email) index	Login case-insensitive y rápido	sql/init.sql:9
events.user_id FK	Propiedad del evento (base anti-IDOR)	sql/init.sql:13
ON DELETE CASCADE	Al borrar un usuario, se borran sus eventos	sql/init.sql:13
idx_events_user_date	Listar/filtrar eventos por usuario y día	sql/init.sql:25

4.3. Conexión y acceso a datos

La conexión se resuelve con un **Pool** de PostgreSQL envuelto en una clase que implementa el **patrón Singleton** (ver sección 7). Además, se registran *type parsers* para que las columnas `DATE` y `TIME` lleguen al cliente como texto ('AAAA-MM-DD' y 'HH:MM'), evitando desajustes de zona horaria que ocurrirían si el driver las convirtiera a objetos Date locales.

```
# backend/src/config/database.js (fragmento)
const DATE_OID = 1082;
const TIME_OID = 1083;
types.setTypeParser(DATE_OID, (value) => value);
types.setTypeParser(TIME_OID, (value) => value ? value.slice(0, 5) : value);
```

El arranque del servidor verifica la conexión antes de aceptar tráfico. Si la base de datos no responde, el proceso termina con código de error, lo que hace visible el problema de inmediato.

```
# backend/src/server.js
async function start() {
  try {
    await db.connect();
    console.log('Base de datos conectada (instancia única / Singleton).');
    app.listen(env.port, () => console.log(`Servidor en http://localhost:${env.port}`));
  } catch (error) {
    console.error('No se pudo iniciar el servidor:', error.message);
    process.exit(1);
  }
}
```

4.4. Acceso parametrizado (prevención de inyección SQL)

Todas las consultas usan **sentencias parametrizadas** ($1, $2, ...), lo que impide la inyección SQL porque los valores nunca se concatenan al texto de la consulta. El acceso está encapsulado en los modelos (`user.model.js` y `event.model.js`). En el caso de eventos, toda consulta está además acotada por `user_id`, que proviene siempre del JWT y nunca del cliente (mitigación de IDOR).

```
# backend/src/models/event.model.js (fragmento anti-IDOR)
static async findByIdForUser(id, userId) {
  const { rows } = await db.query(
    `SELECT ${COLUMNS} FROM events WHERE id = $1 AND user_id = $2`,
    [id, userId]
  );
  return rows[0] ? new Event(rows[0]) : null;
}
```

5. La base de datos: cómo cambiará a futuro

El esquema actual está pensado para el alcance de los Sprints 1 y 2. A medida que el proyecto crezca (Sprints 3 y 4, exportaciones, notificaciones y pruebas), se plantean las siguientes evoluciones. Cada una indica el motivo y el punto del proyecto que se vería afectado.

Evolución	Por qué	Dónde impacta
Migraciones versionadas (node-pg-migrate / Knex / Prisma)	Hoy init.sql solo corre en la primera inicialización; no hay forma de hacer migraciones incrementales ni rollback orde	paquete.sql/ o backend/migrations/
PgBouncer o pool gestionado	El Pool (máx. 10) es suficiente en desarrollo, pero no en producción con múltiples instancias.	backend/src/config/database.js
Tabla refresh_tokens	El JWT vive 1h; para sesiones largas se necesita rotación y revocación de tokens.	backend/src/services/auth.service.js
Soft delete (deleted_at)	Permitir recuperación y auditoría en lugar del borrado físico.	src/event.model.js
Bloqueo optimista (updated_at)	Evitar sobrescrituras cuando dos dispositivos editan el mismo evento.	event.model.js (UPDATE ... WHERE updated_a
Particionado por event_date	Mantener el rendimiento del índice cuando la tabla crezca mucho.	sql/ (PARTITION BY RANGE)
Backups automáticos y cifrado	Continuidad del negocio y protección de datos personales.	Infraestructura / CI-CD
Gestión de secretos (Vault)	No versionar DB_PASSWORD ni JWT_SECRET en archivos de configuración.	backend/src/config/env.js

Nota de compatibilidad: al migrar se deberá preservar el contrato de la API. Por ejemplo, si se agrega `deleted_at`, las consultas de `event.model.js` deberán filtrar `WHERE deleted_at IS NULL` sin cambiar la forma de la respuesta JSON.

6. Sprint 1 — Base móvil, arquitectura y autenticación segura

El objetivo del Sprint 1 fue sentar las bases: conexión a base de datos con el patrón Singleton, un módulo de usuarios con cifrado asíncrono y tokens JWT firmados, una arquitectura limpia de directorios en el frontend, navegación protegida con pilas separadas y formularios de Login/Registro. A continuación se narra qué se hizo y dónde.

6.1. Backend: configuración y seguridad de credenciales

Primero se centralizó la configuración. El módulo de entorno carga el archivo `.env`, expone valores tipados y, en producción, **obliga** a definir un `JWT_SECRET` robusto. Esto evita el error clásico de desplegar con el secreto de desarrollo.

```
# backend/src/config/env.js (fragmento)
if (env.nodeEnv === 'production') {
  required('JWT_SECRET');
  if (env.jwt.secret === 'dev-secret-no-usar-en-produccion')
    throw new Error('JWT_SECRET debe definirse en producción con valor robusto.');
}
```

Luego se implementó el modelo de usuario con **bcrypt**, usando 12 rondas de sal. El correo se normaliza a minúsculas y el método `toJSON()` excluye el hash de la contraseña, de modo que sea imposible filtrarlo por accidente en una respuesta.

```
# backend/src/models/user.model.js (fragmento)
const SALT_ROUNDS = 12;
static async hashPassword(plainPassword) {
  const salt = await bcrypt.genSalt(SALT_ROUNDS);
  return bcrypt.hash(plainPassword, salt);
}
verifyPassword(plainPassword) {
  return bcrypt.compare(plainPassword, this.password);
}
```

6.2. Backend: autenticación con JWT

El servicio de tokens firma y verifica el JWT. El payload solo incluye el identificador y el correo del usuario; la expiración se configura por entorno (por defecto 1h). El middleware de autenticación exige el esquema `Bearer` y adjunta el usuario decodificado en `req.user`, que es la fuente de verdad del propietario en todas las operaciones protegidas.

```
# backend/src/services/token.service.js
function sign(user) {
  return jwt.sign({ sub: user.id, email: user.email },
                  env.jwt.secret, { expiresIn: env.jwt.expiresIn });
}

# backend/src/middleware/auth.middleware.js (fragmento)
const [scheme, token] = header.split(' ');
if (scheme !== 'Bearer' || !token) return res.status(401).json({...});
req.user = tokenService.verify(token);
```

Los controladores de autenticación (`register`, `login`, `me`) validan la entrada, evitan correos duplicados (409), responden 401 en credenciales incorrectas y devuelven siempre `{ user, token }`.

6.3. Frontend: arquitectura limpia y navegación

En el frontend se creó la separación por capas (`domain`, `data`, `navigation`, `presentation`, `services`, `theme`) y la navegación protegida con pilas separadas. El token se guarda con **Expo SecureStore** (Keychain/Keystore), nunca en AsyncStorage. Al arrancar, `AuthContext` restaura la sesión leyendo el token y consultando `/api/auth/me`; si el token es inválido, lo elimina.

```
# frontend/src/services/secureStorage.js (fragmento)
const TOKEN_KEY = 'user_jwt_token';
await SecureStore.setItemAsync(TOKEN_KEY, token);

# frontend/src/presentation/context/AuthContext.js (fragmento)
const token = await getAuthToken();
if (token) setUser(await authRepository.me());
```

Requisito Sprint 1	Implementación	Ubicación
Conexión a BD con Singleton	Clase DatabaseConnection	config/database.js
Cifrado asíncrono de contraseñas	bcrypt (12 rondas)	models/user.model.js
Tokens JWT firmados	sign/verify con expiración	services/token.service.js
Middleware de autenticación	Bearer + req.user	middleware/auth.middleware.js
Arquitectura limpia de directorios	domain/data/navigation/presentation	frontend/src/*
Navegación protegida	AuthStack vs AppStack	navigation/RootNavigator.js
Formularios Login/Registro	Pantallas con validación	presentation/screens/*
JWT en SecureStore	SecureStore (no AsyncStorage)	services/secureStorage.js

7. Patrón Singleton en el proyecto (¿dónde está?)

El enunciado del Sprint 1 pide **configurar la conexión al servidor de base de datos usando el patrón Singleton**. En este proyecto el Singleton está en `backend/src/config/database.js`. El

archivo exporta **una única instancia** de la clase `DatabaseConnection`.

¿Cómo funciona? El constructor comprueba si ya existe una instancia guardada en la propiedad estática `instance`; si no existe, la crea y la guarda, y si existe, la devuelve. De este modo, no importa cuántas veces se haga `require` de este módulo: siempre se obtiene el mismo objeto y, por tanto, el mismo Pool de conexiones.

```
# backend/src/config/database.js (núcleo del Singleton)
class DatabaseConnection {
  constructor() {
    if (!DatabaseConnection.instance) {
      DatabaseConnection.instance = this;
      this.pool = null;
    }
    return DatabaseConnection.instance;   // <-- siempre la misma instancia
  }
  async connect() { /* crea el Pool una sola vez */ }
  getPool() { /* devuelve el Pool o lanza error si no se inicializó */ }
  query(text, params) { return this.getPool().query(text, params); }
}
module.exports = new DatabaseConnection();  // <-- instancia única exportada
```

El Pool se configura con un máximo de 10 conexiones, `idleTimeoutMillis` de 30 s y `connectionTimeoutMillis` de 5 s, y registra un manejador de errores para no tumbar el proceso ante fallos de red.

Aspecto del Singleton	Dónde
Propiedad estática de instancia	database.js:19-23
Creación del Pool única vez	database.js:26-42 (connect)
Exportación de la instancia única	database.js:68
Uso desde el arranque del servidor	server.js:7 (db.connect())
Uso desde los modelos	models/user.model.js y event.model.js (db.query)

8. Arquitectura limpia del frontend

El enunciado también pide **arquitectura limpia de directorios y configuración de navegación activa** en el frontend. La carpeta `frontend/src` está organizada por responsabilidad, de forma que cada capa solo depende de las capas internas.

Capa	Responsabilidad	Ejemplos
domain/	Modelos puros y utilidades (sin React)	entities/Event.js, utils/date.js
data/	Acceso a datos y red	api/client.js, repositories/*
services/	Servicios del dispositivo	secureStorage.js
navigation/	Pilas y contenedor de navegación	RootNavigator, AuthStack, AppStack
presentation/	UI, componentes y estado	screens/*, components/*, context/*
theme/	Tokens de diseño	theme.js

9. Navegación protegida (pilas separadas)

La navegación se resuelve en tres archivos. `RootNavigator` decide qué pila mostrar según el estado de sesión: mientras se restaura el token muestra la `SplashScreen`; si el usuario está autenticado muestra `AppStack` (envuelto en `EventsProvider`); si no, muestra `AuthStack`.

```
# frontend/src/navigation/RootNavigator.js (fragmento)
if (isRestoring) return <SplashScreen />;
return (
  <NavigationContainer theme={navigationTheme}>
    {isAuthenticated
      ? <EventsProvider><AppStack /></EventsProvider>
      : <AuthStack />}
  </NavigationContainer>
);
```

`AuthStack` contiene Login y Register; `AppStack` contiene la Agenda. Esta separación garantiza que un usuario no autenticado **no pueda navegar** a pantallas protegidas, porque ni siquiera están montadas en el árbol de navegación. La condición depende de `useAuth()`, que expone `isAuthenticated` e `isRestoring`.

10. Sprint 2 — Core de la agenda (módulo de eventos)

El Sprint 2 agrega el corazón funcional: el CRUD de eventos protegido por middleware, la vista principal con calendario y la mitigación de IDOR.

10.1. Backend: CRUD protegido y validación estricta

Todas las rutas de eventos se registran detrás del middleware de autenticación, de modo que cualquier petición sin un JWT válido es rechazada antes de llegar al controlador.

```
# backend/src/routes/events.routes.js
router.use(authenticate);          // protege TODAS las rutas de eventos
router.get('/',    eventsController.list);
router.get('/:id', eventsController.getOne);
router.post('/',   eventsController.create);
router.put('/:id', eventsController.update);
router.delete('/:id', eventsController.remove);
```

La validación se hizo con **Zod** en `backend/src/validators/event.schema.js`. Los esquemas usan `.strict()` (rechazan campos no permitidos), expresiones regulares para fecha (AAAA-MM-DD) y hora (HH:MM), enumeraciones para categoría y prioridad, y un `refine` que asegura que la hora de fin sea posterior a la de inicio.

```
# backend/src/validators/event.schema.js (fragmento)
const createEventSchema = withTimeOrder(z.object(baseEvent).strict());
// withTimeOrder -> refine(data => data.startTime < data.endTime)
```

El controlador (`events.controller.js`) valida el cuerpo con `safeParse`, valida el `id` como entero positivo y responde 400 (datos inválidos), 404 (no encontrado) o 201 (creado) según corresponda.

10.2. Backend: mitigación de IDOR (la pieza clave de seguridad)

IDOR (Insecure Direct Object Reference) ocurre cuando un usuario puede acceder a los datos de otro simplemente cambiando un identificador. En este proyecto se mitiga en dos lugares: (1) el controlador toma el propietario de `req.user.sub` (el JWT), nunca del cliente; y (2) el modelo agrega `AND user_id = $n` a toda consulta. Si un usuario intenta leer o modificar el evento de otro, `findByIdForUser` devuelve `null` y el controlador responde 404, sin revelar siquiera que el

recurso existe.

```
# backend/src/controllers/events.controller.js (fragmento)
const event = await Event.findByIdForUser(id.data, req.user.sub);
if (!event) return res.status(404).json({ message: 'Evento no encontrado.' });
```

Evidencia de la mitigación: el comentario en `event.model.js:9-13` documenta explícitamente que el propietario nunca proviene del cliente sino del token JWT.

10.3. Frontend: calendario, modal e interceptores HTTP

En el frontend se implementó la pantalla principal (`AgendaScreen`) con un componente de calendario mensual, la lista de eventos del día seleccionado y un botón flotante para crear. El formulario de creación/edición vive en `EventFormModal`, que valida que el título no esté vacío y que la hora de fin sea posterior a la de inicio.

El estado de los eventos se centraliza en `EventsContext`, que agrupa los eventos por día y expone operaciones `addEvent`, `updateEvent` y `deleteEvent` contra los repositorios.

El requisito de **interceptores HTTP globales que adjunten el JWT automáticamente** se cumple en `data/api/client.js`: un interceptor de petición lee el token de SecureStore y añade la cabecera `Authorization: Bearer ...`; un interceptor de respuesta elimina el token ante un 401, forzando el cierre de sesión.

```
# frontend/src/data/api/client.js (fragmento)
apiClient.interceptors.request.use(async (config) => {
  const token = await getAuthToken();
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
apiClient.interceptors.response.use((r) => r, async (error) => {
  if (error.response?.status === 401) await deleteAuthToken();
  return Promise.reject(error);
});
```

Requisito Sprint 2	Implementación	Ubicación
CRUD de eventos protegido	router.use(authenticate)	routes/events.routes.js
Validación estricta	Zod .strict() + regex + refine	validators/event.schema.js
Mitigación IDOR	user_id desde JWT + WHERE user_id=$n	event.model.js
Vista con calendario	Calendar + AgendaScreen	presentation/components, screens
Interceptores HTTP globales	axios request/response	data/api/client.js

11. Codificación segura aplicada

- **Contraseñas:** bcrypt con 12 rondas de sal; nunca en texto plano.
- **Tokens:** JWT firmado con expiración; guardado en Expo SecureStore (Keychain/Keystore).
- **Inyección SQL:** consultas 100% parametrizadas.
- **Validación:** en cliente (forms) y en servidor (Zod) con rechazo de campos extra.
- **IDOR:** el propietario proviene del token, y toda operación se acota por user_id.
- **Errores:** manejador central (error.middleware.js) que devuelve mensajes controlados.

- **Secretos:** en producción se exige JWT_SECRET robusto (env.js).

12. Comandos utilizados

12.1. Backend

```
cd backend
cp .env.example .env
docker compose up -d      # levanta PostgreSQL (contenedor calendario-db)
npm install
npm run dev               # servidor en http://localhost:4000 (node --watch)

# Verificación rápida del servicio
curl http://localhost:4000/health
```

12.2. Frontend

```
cd frontend
cp .env.example .env      # configurar EXPO_PUBLIC_API_URL
npm install
npx expo start            # inicia el servidor de desarrollo (Metro)
npx expo start --android  # abre en el emulador Android
npx expo lint             # lint
npx tsc --noEmit          # typecheck
npx expo install --fix    # alinear versiones al SDK
npx expo-doctor           # diagnóstico del proyecto
```

12.3. Emulación con Genymotion

Para ejecutar en Genymotion se usó su propio binario `adb` (añadido al PATH) y se configuró la URL de la API al host visto desde la máquina virtual, que en Genymotion es `10.0.3.2` (no localhost).

```
# Añadir el adb de Genymotion al PATH
export PATH="/Applications/Genymotion.app/Contents/MacOS/tools:$PATH"
adb devices               # lista dispositivos (127.0.0.1:6555)

# frontend/.env
EXPO_PUBLIC_API_URL=http://10.0.3.2:4000
```

12.4. Inspección de la base de datos

```
# Consultar datos directamente en el contenedor
docker exec calendario-db psql -U calendario -d calendario -c "SELECT id, email FROM users;"
```

13. Reporte de esta versión del documento

Esta sección documenta qué se agregó a este informe y en qué apartado, para dejar trazabilidad de la propia documentación.

Qué se agregó	Apartado
Sección completa sobre la base de datos: cómo fue realizada (motor, Docker, esquema, índices, type parsers, arranque, consultas p	Sección 4
Sección sobre cómo cambiará la base de datos a futuro (migraciones, PgBouncer, refresh tokens, soft delete, particionado, backups	Sección 5
Narrativa ordenada de los Sprints 1 y 2, indicando archivo y función exacta de cada requisito	Secciones 6 y 10
Ubicación precisa del patrón Singleton (clase DatabaseConnection y líneas).	Sección 7
Mapa de navegación protegida y su justificación de seguridad.	Sección 9

Comandos utilizados (backend, frontend, Genymotion, inspección de BD).	Sección 12
Este mismo reporte de versión.	Sección 13

Herramienta de generación: este PDF se generó con **ReportLab** aplicando la arquitectura **PLATYPUS** (`SimpleDocTemplate`, `Paragraph`, `Table`, `Spacer`, `getSampleStyleSheet()`). El script vive en `documents/generar_documentacion.py`, carpeta ignorada por git por ser de uso interno.

14. Roadmap de evolución de la base de datos

El plan de evolución acompaña a los próximos sprints. En el **Sprint 3** (patrón Strategy para exportación JSON/CSV y notificaciones locales) es probable que no se requieran cambios de esquema, pero sí consultas de agregación. En el **Sprint 4** (pruebas y auditoría) se recomienda incorporar migraciones versionadas y una base de datos de pruebas efímera. A mediano plazo, la prioridad es: (1) migraciones, (2) refresh tokens, (3) soft delete con bloqueo optimista, y (4) observabilidad (métricas de pool y consultas lentas).

```
Corto plazo
  [x] init.sql reproducible vía Docker
  [ ] Migraciones versionadas + entorno de pruebas
Mediano plazo
  [ ] refresh_tokens   [ ] soft delete   [ ] bloqueo optimista
Largo plazo
  [ ] PgBouncer / DB gestionada   [ ] Particionado   [ ] Backups cifrados
```

15. Conclusión

Los Sprints 1 y 2 quedaron implementados de forma coherente con el enunciado: el patrón Singleton está en `config/database.js` y es la única puerta de acceso al Pool de PostgreSQL; la arquitectura limpia y la navegación protegida estructuran el frontend; y la seguridad se apoya en bcrypt, JWT en SecureStore, consultas parametrizadas y la mitigación de IDOR mediante el filtrado por `user_id` tomado del token. La base de datos, hoy reproducible con Docker y un `init.sql`, tiene un camino de evolución claro hacia migraciones versionadas, sesiones con refresh tokens y endurecimiento de la infraestructura.

Este documento es interno y evolutivo: cada vez que se agregue funcionalidad, se añadirá su descripción narrativa, la ubicación exacta en el código y una entrada en el reporte de versión (sección 13).